



Universidad Autónoma del Estado de México
UAEM



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

CENTRO UNIVERSITARIO UAEM “ATLACOMULCO”

Nombre del proyecto

Verificación Vehicular

Paradigmas de la programación

Integrantes

Jorge Luis Gil Bello

Ricardo Morales Flores

Alan Osornio García

Docente: Julio Alberto de la Teja López

GRUPO: ICO 28

Tercer semestre

Licenciatura en Ingeniería en Computación.

Plan de Estudios: F19



Índice

Tabla de contenido

Índice	2
Introducción.....	3
Propósito del Proyecto.....	4
Funcionalidades Principales.....	5
Componentes del Proyecto.....	6
Diagramas	7
.....	9
Códigos en Python	12
Diseño	37
.....	38
.....	38
Resultado	39
Rol de cada integrante	46
PythonAnywhere	47
GitHub.....	49
Conclusión.....	50



Introducción

Este proyecto consiste en una aplicación web desarrollada en Python utilizando el framework Flask, que combina diversas funcionalidades relacionadas con la gestión de vehículos. El sistema está diseñado para ofrecer herramientas útiles como la verificación de restricciones de circulación (basadas en el programa "Hoy No Circula") y el registro de citas vehiculares, integrando una base de datos para almacenar información relevante.

El sistema cuenta con una interfaz gráfica diseñada para ser intuitiva y estéticamente agradable, asegurando una experiencia de usuario óptima. Además, incorpora un módulo que implementa la lógica necesaria para mostrar las restricciones vehiculares según los días de la semana, colores de engomado y terminación de placas. La aplicación también incluye un calendario interactivo que permite a los usuarios visualizar y seleccionar días específicos, registrando esta información en una base de datos que gestiona eficazmente los datos almacenados, garantizando su correcto acceso, actualización y recuperación.

Asimismo, el sistema integra un panel de administrador que permite gestionar el sistema de forma centralizada. Este panel incluye funcionalidades avanzadas como la administración de usuarios, la visualización de estadísticas relacionadas con las citas y restricciones vehiculares, y la actualización de parámetros del programa "Hoy No Circula".



Propósito del Proyecto

Este sistema combina herramientas prácticas para la gestión vehicular con una interfaz amigable e intuitiva, permitiendo a los usuarios:

- Informarse sobre las restricciones y requisitos de verificación vehicular.
- Registrar citas relacionadas con vehículos de forma rápida y eficiente.
- Visualizar calendarios útiles para la planificación.



Funcionalidades Principales

1. Verificación de "Hoy No Circula":

- Los usuarios pueden ingresar la placa de un vehículo y el día de la semana para determinar si dicho vehículo puede circular.
- Se utiliza el último dígito de la placa y un conjunto de restricciones predefinidas según el día.

2. Registro de Citas Vehiculares:

- Los usuarios pueden registrar citas proporcionando información del vehículo, como placa, serie, modelo y correo electrónico.
- Los datos se almacenan en una base de datos MySQL mediante una clase personalizada para la gestión de datos.

3. Generación y Visualización de Calendarios:

- El proyecto incluye una funcionalidad adicional para generar calendarios mensuales, que pueden ser útiles para la gestión de citas u otras funcionalidades relacionadas.



Componentes del Proyecto

1. **Flask:**

- Maneja las rutas de la aplicación, como la página principal, la verificación vehicular y el registro de citas.
- Utiliza plantillas HTML para proporcionar una interfaz dinámica al usuario.

2. **Base de Datos:**

- Implementada mediante MySQL y gestionada con la clase Database para insertar y consultar información.

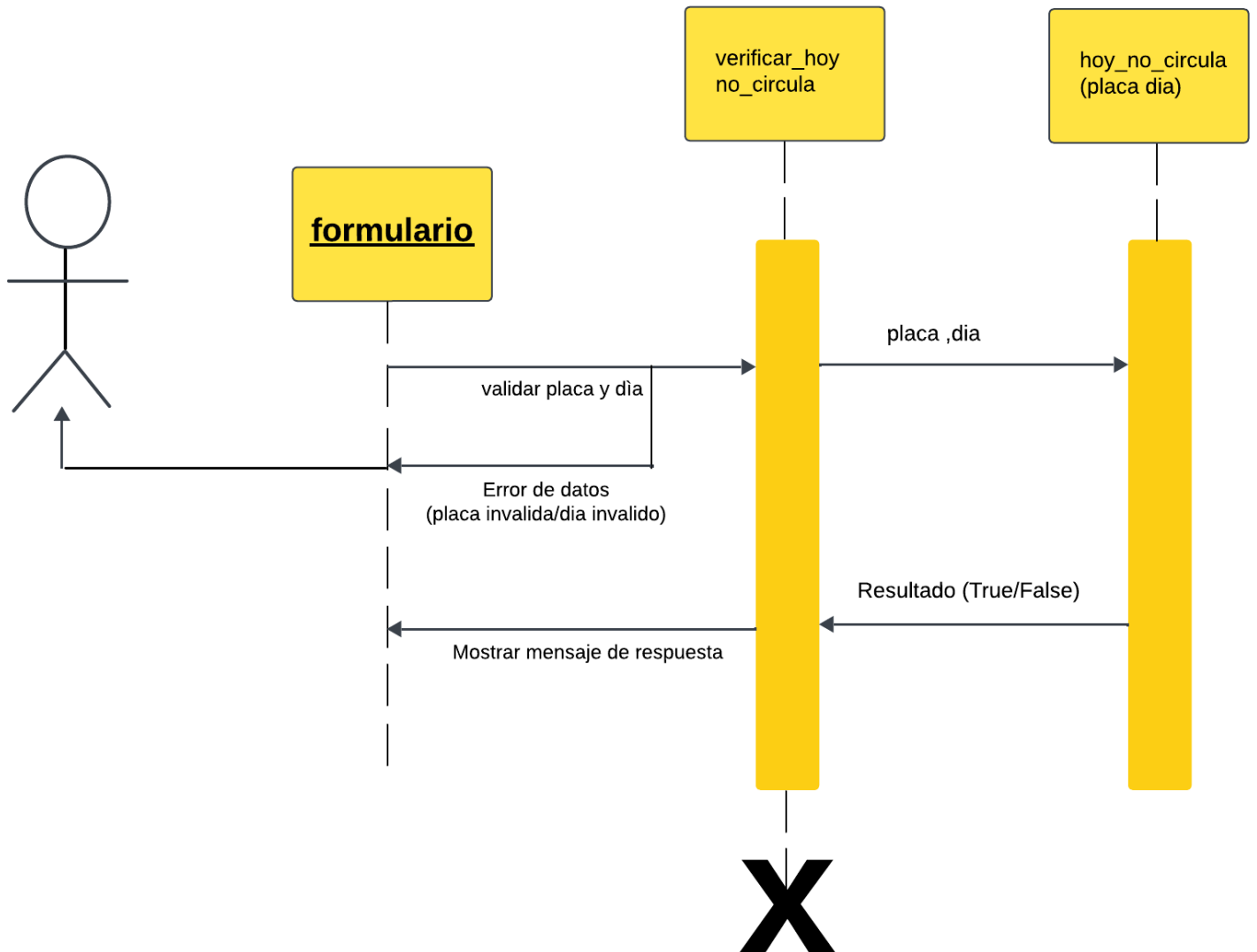
3. **Lógica del Programa:**

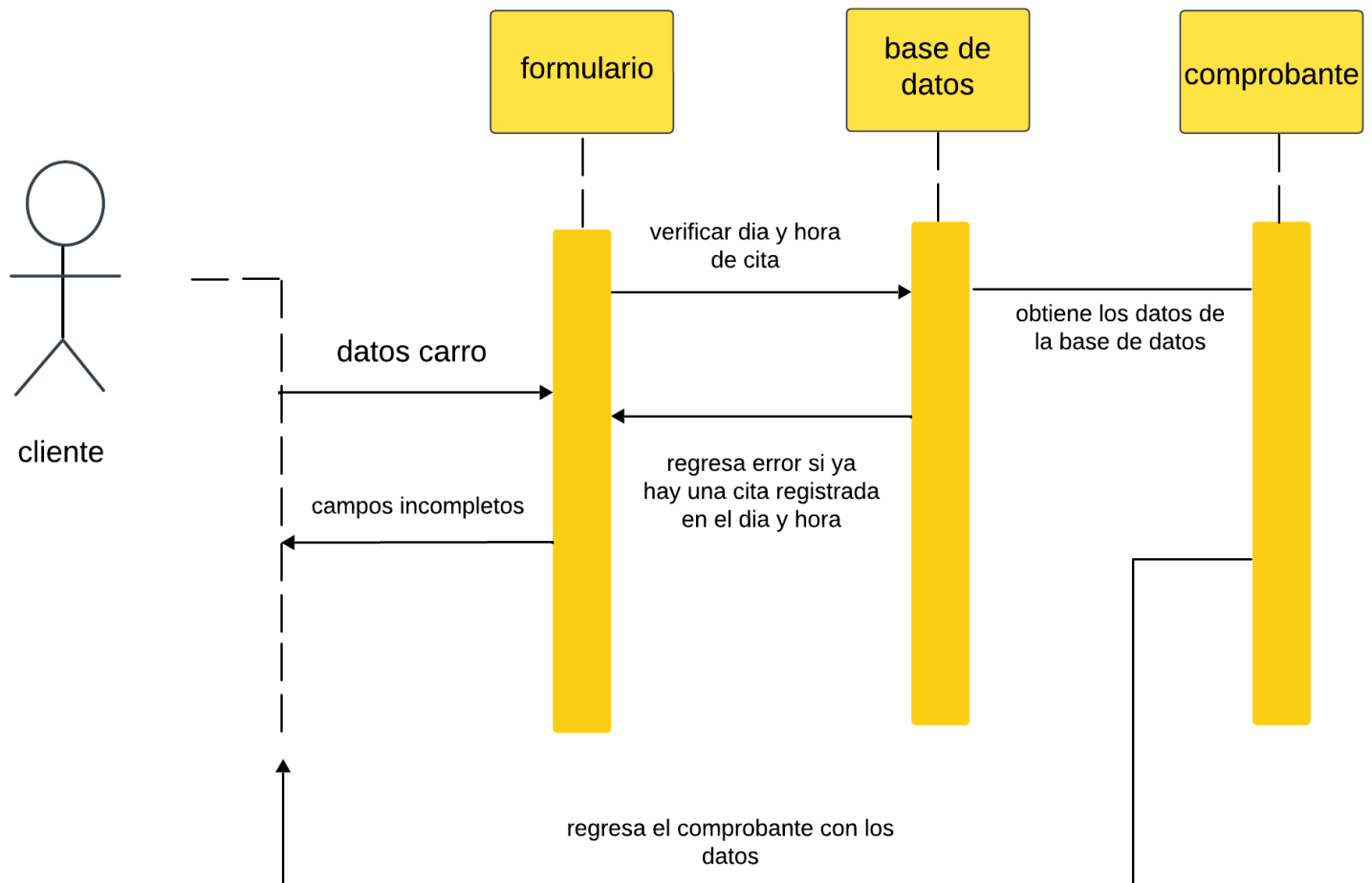
- La lógica de verificación vehicular analiza las placas de los vehículos para determinar las restricciones y requisitos establecidos.
- El módulo de calendario permite generar representaciones mensuales estructuradas, ideales para la gestión de citas.



Diagramas

Diagramas de Secuencia





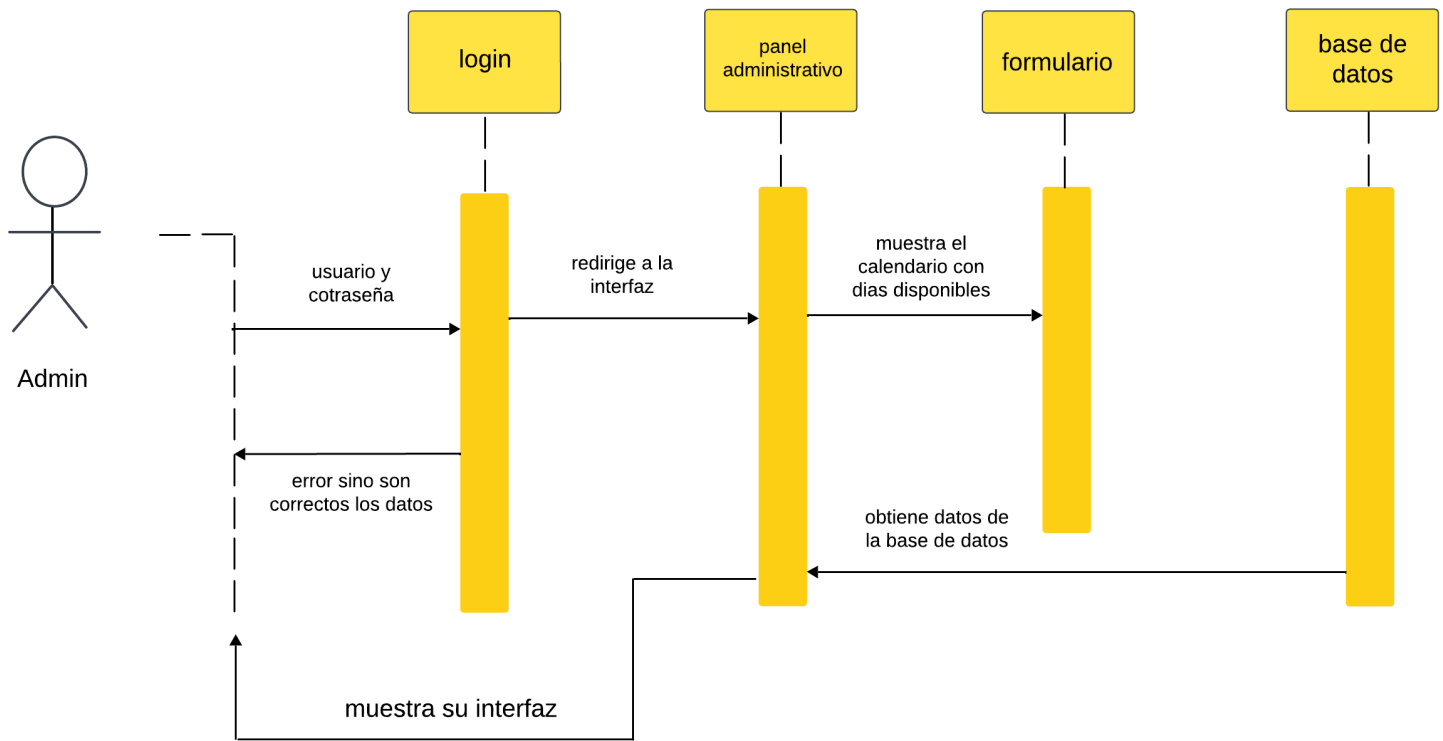




Diagrama de casos de uso

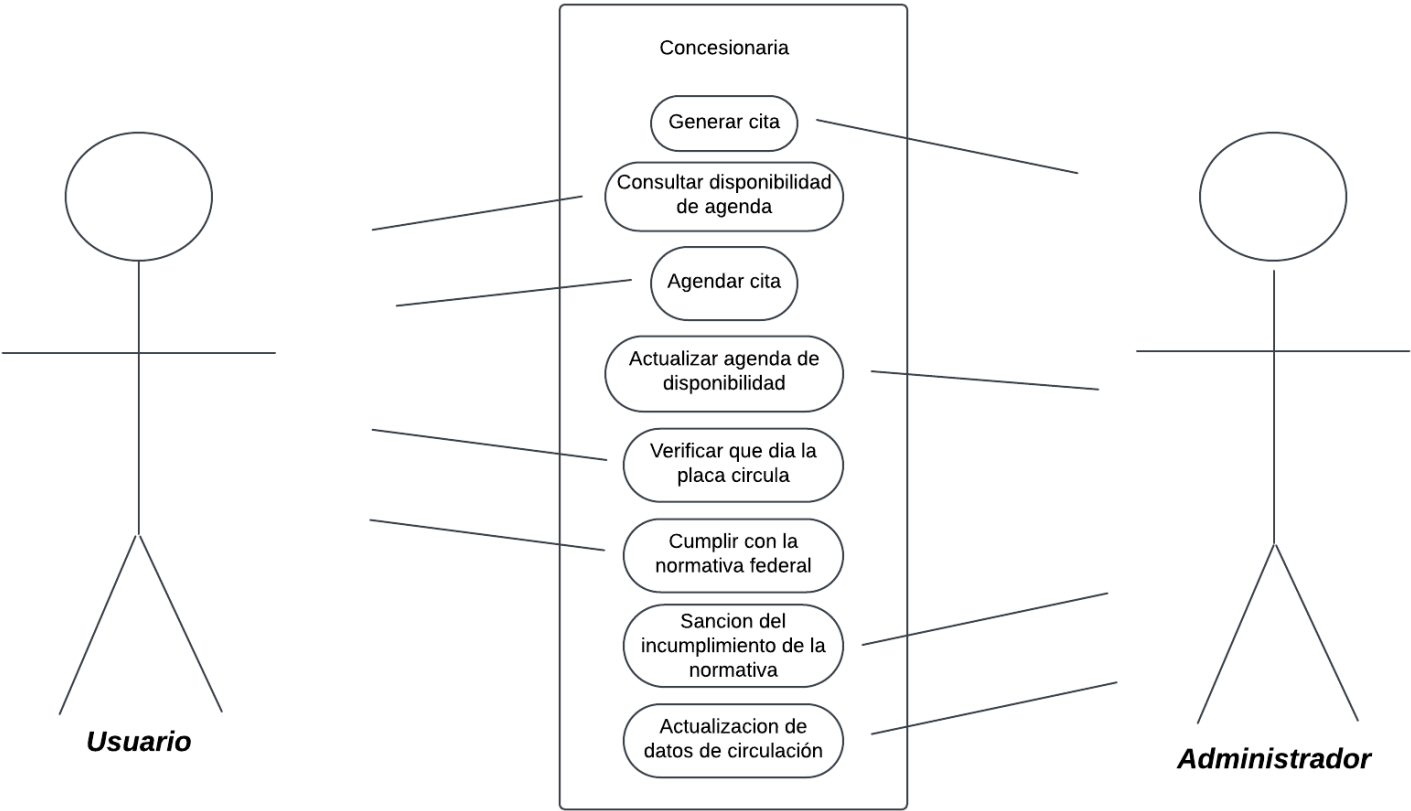
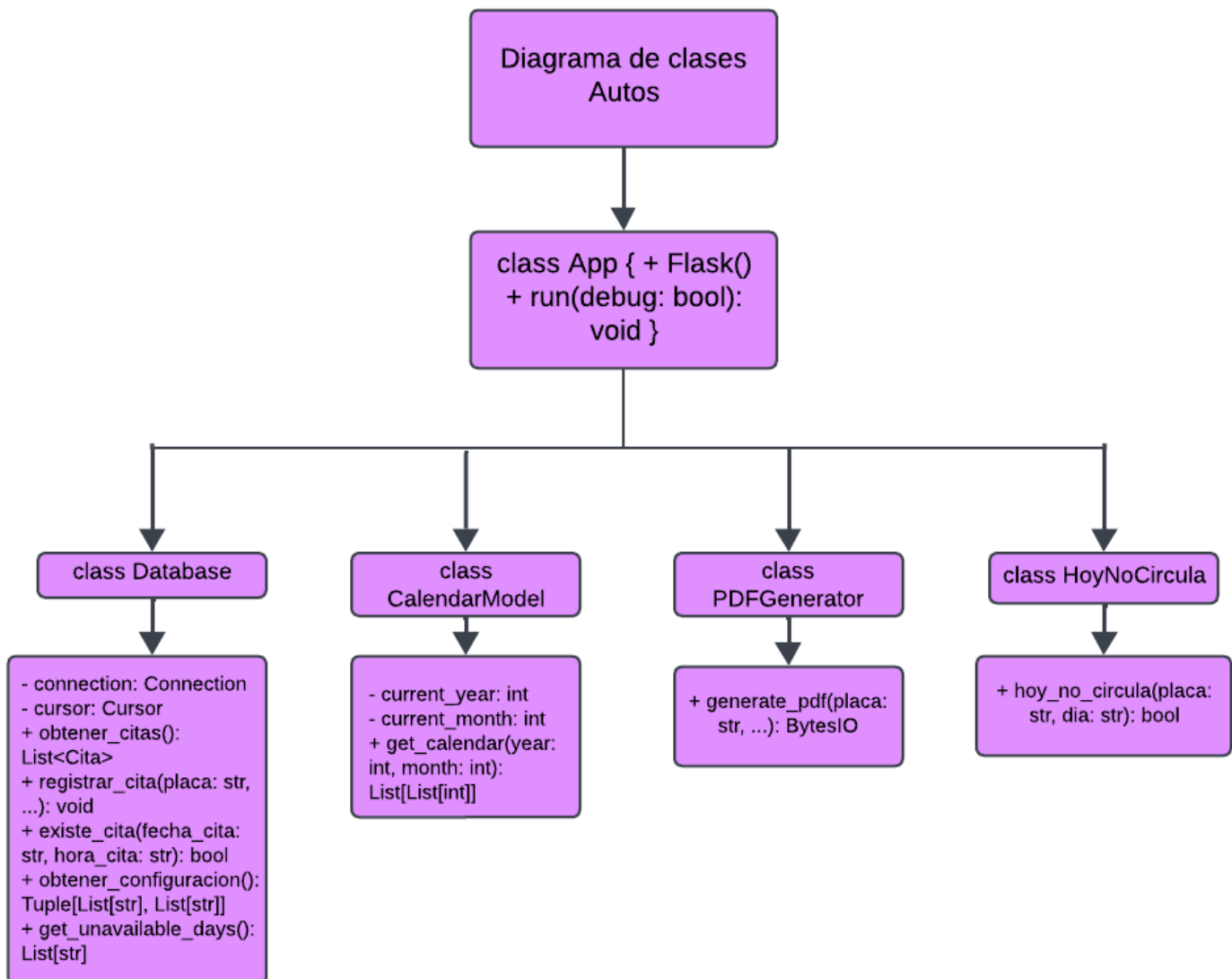




Diagrama de clases





Códigos en Python

App.py

```
from flask import Flask, render_template, request, flash, redirect, url_for
from hoy_no_circula import hoy_no_circula
from database import Database
import pymysql
from reportlab.pdfgen import canvas
from datetime import datetime
import io
from flask import send_file
from reportlab.lib.pagesizes import letter
from reportlab.lib.colors import black, lightgrey
app = Flask(__name__)
app.secret_key = 'tu_clave_secreta'
def get_db_connection():
    return pymysql.connect(
        host="localhost",
        user="root",
        password="",
        database="registroautos"
    )
@app.route('/registros')
def registros_autos():
    connection = get_db_connection()
    cursor = connection.cursor(pymysql.cursors.DictCursor)
    cursor.execute("SELECT * FROM registros")
    registros = cursor.fetchall()
    connection.close()
    return render_template('admin_panel.html', registros=registros)
@app.route('/')
def index():
    dia = ""
    if request.method == "POST":
        placa = request.form.get("placa")
        dia = request.form.get("dia")
        if placa and dia:
            dia = hoy_no_circula(placa, dia)
    return render_template("index.html", dia=dia)
@app.route("/administrativo")
def administrativo_view():
    return render_template("login.html")
```



```
@app.route("/login_action", methods=["POST"])
def login_action():
    username = request.form.get("username")
    password = request.form.get("password")
    if username == "admin" and password == "1234":
        db = Database()
        registros = db.obtener_citas()
        return render_template("admin_panel.html", registros=registros)
    return "Usuario o contraseña incorrectos", 401

@app.route('/admin')
def admin_panel():
    db = Database()
    registros = db.obtener_citas()
    print("Registros obtenidos:", registros)
    return render_template('admin_panel.html', registros=registros)

@app.route('/registro_cita', methods=['GET', 'POST'])
def registro_cita():
    if request.method == 'POST':
        placa = request.form.get("placa")
        confirm_placa = request.form.get("confirm_placa")
        serie = request.form.get("serie")
        confirm_serie = request.form.get("confirm_serie")
        modelo = request.form.get("modelo")
        correo_electronico = request.form.get("correo_electronico")
        fecha_cita = request.form.get("fecha_cita")
        hora_cita = request.form.get("hora_cita")
        if not all([placa, confirm_placa, serie, confirm_serie, modelo, correo_electronico,
        fecha_cita, hora_cita]):
            flash("Por favor, completa todos los campos obligatorios.", "error")
            return redirect(url_for("registro_cita"))
        if placa != confirm_placa:
            flash("Las placas no coinciden.", "error")
            return redirect(url_for("registro_cita"))
        if serie != confirm_serie:
            flash("Las series no coinciden.", "error")
            return redirect(url_for("registro_cita"))
        if db.existe_cita(fecha_cita, hora_cita):
            flash("La cita ya existe para esta fecha y hora.", "error")
            return redirect(url_for("registro_cita"))
        db.registrar_cita(placa, confirm_placa, serie, confirm_serie, modelo,
        correo_electronico, fecha_cita, hora_cita)
        pdf_buffer = generate_pdf(placa, serie, modelo, fecha_cita, hora_cita)
        flash("Cita registrada con éxito. Se ha generado el comprobante.", "success")
        return send_file(
```



```
pdf_buffer,  
mimetype="application/pdf",  
as_attachment=True,  
download_name="comprobante_cita.pdf"  
)  
dias_disponibles, horarios_disponibles = db.obtener_configuracion()  
return render_template("registro_cita.html", dias_disponibles=dias_disponibles,  
horarios_disponibles=horarios_disponibles)  
def generate_pdf(placa, serie, modelo, fecha_cita, hora_cita):  
    buffer = io.BytesIO()  
    c = canvas.Canvas(buffer, pagesize=letter)  
    width, height = letter  
    margin_x = 50  
    title_y = height - 80  
    section_start_y = title_y - 50  
    line_height = 20  
    now = datetime.now()  
    fecha_registro = now.strftime("%Y-%m-%d")  
    hora_registro = now.strftime("%H:%M:%S")  
    c.setFont("Helvetica-Bold", 30)  
    c.setFillColor(black)  
    c.drawCentredString(width / 2, title_y, "Comprobante de Cita")  
    c.setLineWidth(1)  
    c.setStrokeColor(lightgrey)  
    c.line(margin_x, title_y - 10, width - margin_x, title_y - 10)  
    c.setFont("Helvetica", 12)  
    c.setFillColor(black)  
    c.drawString(margin_x, section_start_y, "Detalles de la Cita:")  
    section_start_y -= line_height  
    c.drawString(margin_x + 20, section_start_y, f"Placa: {placa}")  
    section_start_y -= line_height  
    c.drawString(margin_x + 20, section_start_y, f"Serie: {serie}")  
    section_start_y -= line_height  
    c.drawString(margin_x + 20, section_start_y, f"Modelo: {modelo}")  
    section_start_y -= line_height  
    c.drawString(margin_x + 20, section_start_y, f"Fecha de la Cita: {fecha_cita}")  
    section_start_y -= line_height  
    c.drawString(margin_x + 20, section_start_y, f"Hora de la Cita: {hora_cita}")  
    section_start_y -= 20  
    c.line(margin_x, section_start_y, width - margin_x, section_start_y)  
    section_start_y -= line_height  
    c.drawString(margin_x, section_start_y, "Registro de Cita:")  
    section_start_y -= line_height  
    c.drawString(margin_x + 20, section_start_y, f"Fecha de Registro: {fecha_registro}")
```



```
section_start_y -= line_height
c.drawString(margin_x + 20, section_start_y, f"Hora de Registro: {hora_registro}")
c.setFont("Helvetica-Oblique", 10)
footer_y = 40
c.drawCentredString(width / 2, footer_y, "Gracias por usar nuestro servicio.")
c.showPage()
c.save()
buffer.seek(0)
return buffer

@app.route('/verificar_hoy_no_circula', methods=['POST'])
def verificar_hoy_no_circula():
    placa = request.form.get('placa')
    dia = request.form.get('dia').lower()
    if not placa or not dia:
        return "Por favor, proporciona la placa y el día para verificar."
    resultado = hoy_no_circula(placa, dia)
    if resultado is None:
        return "El día seleccionado no es válido o la placa proporcionada no es válida."
    elif resultado:
        return f"La placa {placa} **NO** puede circular el día {dia.capitalize()}."
    else:
        return f"La placa {placa} **SÍ** puede circular el día {dia.capitalize()}."

@app.route('/hoy_no_circula')
def hoy_no_circula_view():
    return render_template('hoy_no_circula.html', message=None)

@app.route('/comprobante/<int:id_cita>')
def comprobante(id_cita):
    return send_file(f"ruta/del/comprobante_{id_cita}.pdf", as_attachment=False)

db = Database()

@app.route('/registrar', methods=['GET', 'POST'])
def registrar():
    if request.method == 'POST':
        placa = request.form['placa']
        confirm_placa = request.form['confirm_placa']
        serie = request.form['serie']
        confirm_serie = request.form['confirm_serie']
        modelo = request.form['modelo']
        correo_electronico = request.form['correo_electronico']
        fecha_cita = request.form['fecha_cita']
        hora_cita = request.form['hora_cita']
        if not all([placa, confirm_placa, serie, confirm_serie, modelo, correo_electronico,
        fecha_cita, hora_cita]):
            flash('Por favor, completa todos los campos obligatorios.', 'error')
            return redirect(url_for('registrar'))
```



```
if placa != confirm_placa:
    flash('Las placas no coinciden.', 'error')
    return redirect(url_for('registrar'))
if serie != confirm_serie:
    flash('Las series no coinciden.', 'error')
if db.existe_cita.fecha_cita, hora_cita):
    flash('La cita ya existe para esta fecha y hora.', 'error')
    return redirect(url_for('registrar'))
db.registrar_cita(placa, confirm_placa, serie, confirm_serie, modelo,
correo_electronico, fecha_cita, hora_cita)
flash('Cita registrada con éxito.', 'success')
return redirect(url_for('registrar'))
dias_disponibles, horarios_disponibles = db.obtener_configuracion()
return render_template('registro_cita.html', dias_disponibles=dias_disponibles,
horarios_disponibles=horarios_disponibles)
if __name__ == "__main__":
    app.run(debug=True)
```

ACTUALIZACIONES

```
@app.route('/crear_usuario', methods=['GET'])
def mostrar_formulario_crear_usuario():
    return render_template('crear_usuario.html')

@app.route('/crear_usuario', methods=['POST'])
def crear_usuario():
    nombre_usuario = request.form['nombre_usuario']
    correo = request.form['correo']
    contrasena = request.form['contrasena']

    db = Database()
    db.crear_usuario(nombre_usuario, correo, contrasena)
```




```
flash("Usuario creado con éxito.", "success")

return redirect(url_for('iniciar_sesion'))

@app.route('/iniciar_sesion', methods=['GET', 'POST'])
def iniciar_sesion():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        session['usuario'] = username # Guardar al usuario en la sesión
        return redirect(url_for('home')) # Página principal después de iniciar sesión
    return render_template('iniciar_sesion.html')

@app.route('/logout')
def logout():
    session.pop('usuario', None) # Eliminar la sesión del usuario
    flash("Has cerrado sesión correctamente.", "success")
    return redirect(url_for('index')) # Redirigir al index

@app.route('/home')
def home():
    if 'usuario' not in session: # Verificar si el usuario está autenticado
        return redirect(url_for('iniciar_sesion')) # Si no, redirigir al login
    return render_template('index.html') # Página principal

@app.route('/modificar_cita/<string:cita_id>', methods=['GET', 'POST'])
def modificar_cita(cita_id):
    db = Database()
    cita = db.obtener_cita_por_placa(cita_id) # Obtenemos la cita por la placa

    if not cita:
```



```
flash("Cita no encontrada.", "error")  
return redirect(url_for('registros_autos'))
```

```
if request.method == 'POST':  
    placa = request.form['placa']  
    serie = request.form['serie']  
    modelo = request.form['modelo']  
    correo_electronico = request.form['correo_electronico']  
    fecha_cita = request.form['fecha_cita']  
    hora_cita = request.form['hora_cita']  
  
    # Llamamos a la función para actualizar la cita  
    db.actualizar_cita(placa, serie, modelo, correo_electronico, fecha_cita, hora_cita)  
  
    # Redirigimos a la página de registros después de la actualización  
    flash('Cita modificada con éxito.', 'success')  
    return redirect(url_for('registros_autos'))  
  
return render_template('modificar_cita.html', cita=cita)  
  
if __name__ == '__main__':  
    app.run(debug=True)
```



Explicación

1. Propósito General

Este código implementa una aplicación web para la gestión de citas de verificación vehicular, proporcionando funcionalidades para registrar citas, administrar registros y generar comprobantes en PDF, todo integrado con una base de datos MySQL.

2. Estructura del Proyecto

a) Módulo Principal:

- Configura el servidor Flask, define rutas, y gestiona el flujo de trabajo de la aplicación.
- Maneja interacciones del usuario a través de formularios web y genera respuestas dinámicas.

b) Base de Datos:

- Las citas y registros se almacenan en una base de datos MySQL.
- Se utiliza una clase Database para interactuar con la base de datos, realizar consultas y verificar datos.

c) Generación de PDF:

- Cuando un usuario registra una cita, el sistema genera un comprobante en formato PDF usando la biblioteca ReportLab.
- Este comprobante contiene los detalles del vehículo, la cita y la fecha de registro.

d) Gestión de Restricciones:

- El sistema verifica si un vehículo puede circular en base a restricciones predefinidas según la placa y el día.



3. Funcionalidades Principales

a) Registro de Citas Vehiculares:

- Los usuarios pueden registrar una cita proporcionando detalles del vehículo (placa, serie, modelo, etc.).
- El sistema valida que los datos ingresados sean correctos y que no existan citas duplicadas en la misma fecha y hora.

b) Generación y Descarga de Comprobantes:

- Una vez registrada la cita, se genera un PDF con el comprobante que los usuarios pueden descargar.

c) Panel Administrativo:

- Los administradores pueden acceder a un panel para visualizar y gestionar las citas registradas.
- Incluye una funcionalidad de autenticación básica con credenciales de administrador.

d) Validación de Circulación Vehicular:

- Permite verificar si un vehículo tiene permitido circular en un día específico según las restricciones basadas en el último dígito de la placa.

e) Visualización de Registros:

- Muestra una tabla con todos los registros almacenados en la base de datos en una vista dedicada.



4.Descripción Técnica

a) Rutas y Lógica de Flask:

- Las rutas como /registrar, /admin, y /registros manejan las acciones de usuario y renderizan las plantillas correspondientes.
- Formularios HTML envían datos al servidor para procesarlos y generar respuestas dinámicas.

b) Interacción con la Base de Datos:

- Se utiliza la biblioteca pymysql para conectarse y realizar operaciones en la base de datos MySQL.

c) Generación de PDF:

- El comprobante de cita se genera mediante ReportLab, personalizando el diseño para incluir información clave del vehículo y la cita.

d) Validaciones y Mensajes:

- Se utilizan validaciones para asegurar que los datos ingresados sean correctos (coincidencia de placas, series, campos obligatorios, etc.).
- Los mensajes de error o éxito se muestran al usuario mediante flash.

e) Interfaz

- Las plantillas HTML están diseñadas para ser intuitivas y accesibles, permitiendo a los usuarios navegar por el sistema sin complicaciones.

5.Aplicaciones

- Registrar y gestionar citas de verificación vehicular.
- Generar comprobantes automatizados.
- Administrar datos de clientes y vehículos.



ACTUALIZACIONES.

1. Ruta `@app.route('/crear_usuario', methods=['GET'])`:
 - a. Muestra un formulario (`crear_usuario.html`) para crear un usuario.
 - b. El método es GET, por lo que solo renderiza la plantilla HTML.
2. Ruta `@app.route('/crear_usuario', methods=['POST'])`:
 - a. Procesa los datos enviados desde el formulario para crear un nuevo usuario.
 - b. Obtiene los datos del formulario (`nombre_usuario`, `correo`, `contrasena`) a través de `request.form`.
 - c. Utiliza una clase `Database` (no definida aquí) para guardar el usuario en la base de datos.
 - d. Usa flash para mostrar un mensaje de éxito en la interfaz.
 - e. Redirige a la página de inicio de sesión (`iniciar_sesion`).
3. Iniciar Sesión
4. Ruta `@app.route('/iniciar_sesion', methods=['GET', 'POST'])`:
 - a. Para el método GET, muestra el formulario de inicio de sesión (`iniciar_sesion.html`).
 - b. Para el método POST, procesa los datos enviados por el formulario.
 - i. Obtiene el nombre de usuario (`username`) y contraseña (`password`) del formulario.
 - ii. Guarda el nombre de usuario en `session`, lo que permite mantener la sesión del usuario.
 - iii. Redirige a la página principal (`home`) después del inicio de sesión.
5. Cerrar Sesión
6. Ruta `@app.route('/logout')`:
 - a. Elimina la sesión del usuario (`session.pop`).
 - b. Muestra un mensaje de confirmación de cierre de sesión con flash.



- c. Redirige al índice (index).

7. Página Principal

8. Ruta @app.route('/home'):

- a. Verifica si el usuario está autenticado ('usuario' in session).
- b. Si no está autenticado, redirige al formulario de inicio de sesión (iniciar_sesion).
- c. Si está autenticado, renderiza la página principal (index.html).

9. Modificar Cita

10. Ruta @app.route('/modificar_cita/<string:cita_id>', methods=['GET', 'POST']):

- a. La ruta incluye un parámetro dinámico cita_id, que representa el identificador de la cita.
- b. Al realizar una solicitud GET:
 - i. Obtiene la cita correspondiente usando el método obtener_cita_por_placa de la clase Database.
 - ii. Si no se encuentra la cita, muestra un mensaje de error y redirige a registros_autos.
 - iii. Si se encuentra la cita, renderiza el formulario para modificarla (modificar_cita.html).
- c. Al realizar una solicitud POST:
 - i. Obtiene los datos modificados desde el formulario.
 - ii. Llama al método actualizar_cita para guardar los cambios en la base de datos.
 - iii. Muestra un mensaje de éxito y redirige a la página de registros (registros_autos).

11. Facilitar procesos administrativos a través de un panel dedicado.

Database.py

```
from flask import flash
import pymysql

class Database:
```



```
def __init__(self, host="localhost", user="root", password="", database="registroautos"):
    self.connection = pymysql.connect(
        host=host,
        user=user,
        password=password,
        database=database
    )
    self.cursor = self.connection.cursor()
def obtener_citas(self):
    try:
        query = """
            SELECT Placa, Serie, Modelo, Correo_Electronico, Fecha_Cita, Hora_Cita
            FROM registros
        """
        self.cursor.execute(query)
        citas = self.cursor.fetchall()
        registros = [
            {
                'Placa': cita[0],
                'Serie': cita[1],
                'Modelo': cita[2],
                'Correo_Electronico': cita[3],
                'Fecha_Cita': cita[4],
                'Hora_Cita': cita[5]
            }
            for cita in citas
        ]
        return registros
    except Exception as e:
        print("Error al obtener citas:", e)
        return []
def obtener_configuracion(self):
    dias_disponibles = ["Lunes", "Miércoles", "Viernes"]
    horarios_disponibles = ["09:00", "10:00", "12:00"]
    return dias_disponibles, horarios_disponibles
def obtener_horas_ocupadas(self, fecha_cita):
    query = """
        SELECT Hora_Cita FROM registros WHERE Fecha_Cita = %s
    """
    self.cursor.execute(query, (fecha_cita,))
    ocupadas = self.cursor.fetchall()
    return [hora[0] for hora in ocupadas]
def registrar_cita(self, placa, confirm_placa, serie, confirm_serie, modelo,
correo_electronico, fecha_cita, hora_cita):
```




```
query = """
INSERT INTO registros (Placa, Confirmar_Placa, Serie, Confirmar_Serie, Modelo,
Correo_Electronico, Fecha_Cita, Hora_Cita)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
"""

self.cursor.execute(query, (placa, confirm_placa, serie, confirm_serie, modelo,
correo_electronico, fecha_cita, hora_cita))
self.connection.commit()

def existe_cita(self, fecha_cita, hora_cita):
    query = """
    SELECT COUNT(*) FROM registros WHERE Fecha_Cita = %s AND Hora_Cita = %s
    """

    self.cursor.execute(query, (fecha_cita, hora_cita))
    result = self.cursor.fetchone()
    return result[0] > 0

def get_unavailable_days(self):
    query = "SELECT fecha FROM dias_no_disponibles"
    self.cursor.execute(query)
    return [row[0] for row in self.cursor.fetchall()]

def get_unavailable_hours(self, fecha):
    query = "SELECT hora FROM horas_no_disponibles WHERE fecha = %s"
    self.cursor.execute(query, (fecha,))
    return [row[0] for row in self.cursor.fetchall()]

def insert_unavailable_day(self, fecha):
    query = "INSERT INTO dias_no_disponibles (fecha) VALUES (%s)"
    self.cursor.execute(query, (fecha,))
    self.connection.commit()

def insert_unavailable_hour(self, fecha, hora):
    query = "INSERT INTO horas_no_disponibles (fecha, hora) VALUES (%s, %s)"
    self.cursor.execute(query, (fecha, hora))
    self.connection.commit()
```

Actualizaciones.

```
def actualizar_cita(self, placa, nueva_serie, nuevo_modelo, nuevo_correo, nueva_fecha_cita, nueva_hora_cita):
```

```
    try:
```

```
        query = """
```



UPDATE registros

SET Serie = %s, Modelo = %s, Correo_Electronico = %s, Fecha_Cita = %s, Hora_Cita = %s

WHERE Placa = %s

"""

self.cursor.execute(query, (nueva_serie, nuevo_modelo, nuevo_correo, nueva_fecha_cita, nueva_hora_cita, placa))

self.connection.commit()

except Exception as e:

print(f"Error al actualizar cita: {e}")

self.connection.rollback()

def registrar_usuario(self, nombre_usuario, correo, contrasena):

"""

Registra un nuevo usuario en la base de datos.

"""

try:

if self.usuario_existe(correo):

flash("El correo ya está registrado", "error")

return False

query = """

INSERT INTO usuarios (nombre_usuario, correo, contrasena)

VALUES (%s, %s, %s)

"""

self.cursor.execute(query, (nombre_usuario, correo, contrasena))

self.connection.commit()

return True

except Exception as e:

flash(f"Error al registrar usuario: {e}", "error")

return False



```
def autenticar_usuario(self, correo, contrasena):
```

```
    """
```

```
    Verifica las credenciales del usuario.
```

```
    """
```

```
    try:
```

```
        query = """
```

```
        SELECT id, nombre_usuario FROM usuarios
```

```
        WHERE correo = %s AND contrasena = %s
```

```
        """
```

```
        self.cursor.execute(query, (correo, contrasena))
```

```
        usuario = self.cursor.fetchone()
```

```
        if usuario:
```

```
            return {"id": usuario[0], "nombre_usuario": usuario[1]}
```

```
        return None
```

```
    except Exception as e:
```

```
        flash(f"Error al autenticar usuario: {e}", "error")
```

```
        return None
```

```
def usuario_existe(self, correo):
```

```
    """
```

```
    Verifica si un correo ya está registrado.
```

```
    """
```

```
    try:
```

```
        query = """
```

```
        SELECT COUNT(*) FROM usuarios WHERE correo = %s
```

```
        """
```

```
        self.cursor.execute(query, (correo,))
```

```
        result = self.cursor.fetchone()
```

```
        return result[0] > 0
```

```
    except Exception as e:
```



```
flash(f"Error al verificar existencia de usuario: {e}", "error")  
  
return False
```

Método para crear usuario

```
def crear_usuario(self, nombre_usuario, correo, contraseña):
```

```
    try:
```

```
        sql = "INSERT INTO usuarios (nombre_usuario, correo, contraseña) VALUES (%s, %s, %s)"
```

```
        self.cursor.execute(sql, (nombre_usuario, correo, contraseña))
```

```
        self.connection.commit()
```

```
    except Exception as e:
```

```
        print(f"Error al crear usuario: {e}")
```

```
        self.connection.rollback()
```

Explicación

1. Propósito General

La clase Database es responsable de gestionar la interacción entre la aplicación y la base de datos MySQL. Está diseñada para manejar registros relacionados con citas vehiculares, así como para realizar operaciones relacionadas con días y horarios no disponibles.

2. Métodos Principales

a) Obtener Citas

- Recupera las citas registradas en la base de datos desde la tabla registros.
- Salida: Una lista de diccionarios con detalles de las citas (Placa, Serie, Modelo, etc.).
- Error Handling: Captura excepciones y devuelve una lista vacía en caso de error

b) Obtener Configuración de Días y Horarios Disponibles

- Devuelve listas predefinidas de días y horarios disponibles para agendar citas.

c) Obtener Horas Ocupadas



- Recupera todas las horas ya ocupadas para una fecha específica.

d) Registrar una Nueva Cita

- Inserta una nueva cita en la tabla registros.
- Validación Previa: Se recomienda verificar conflictos de horarios usando `existe_cita()` antes de llamar a este método.

e) Verificar Existencia de una Cita

- Comprueba si ya existe una cita para una combinación de fecha y hora.
- Salida: Devuelve True si existe al menos una cita; de lo contrario, False.

3. Gestión de Días y Horarios No Disponibles

a) Obtener Días No Disponibles

- Recupera los días no disponibles almacenados en la tabla `dias_no_disponibles`.

b) Obtener Horas No Disponibles

- Recupera las horas no disponibles para una fecha específica desde la tabla `horas_no_disponibles`.

c) Insertar un Día No Disponible

- Agrega un nuevo día no disponible en la tabla `dias_no_disponibles`.

d) Insertar una Hora No Disponible

- Agrega una hora específica como no disponible en una fecha determinada.



Actualizaciones.

1. Método actualizar_cita

Este método actualiza los datos de una cita en la base de datos.

- Parámetros:
 - placa: Identificador de la cita.
 - nueva_serie, nuevo_modelo, nuevo_correo, nueva_fecha_cita, nueva_hora_cita: Nuevos valores para actualizar la cita.
- Funcionamiento:
 - Ejecuta un comando SQL UPDATE que modifica los datos de la cita según la placa proporcionada.
 - Usa commit() para confirmar los cambios en la base de datos.
 - Si ocurre un error, captura la excepción, imprime el error, y revierte los cambios con rollback().

2. Método registrar_usuario

Registra un nuevo usuario en la base de datos.

- Funcionamiento:
 - Comprueba si el correo ya está registrado llamando al método usuario_existe.
 - Si el correo no existe, inserta un nuevo usuario en la tabla usuarios con los datos proporcionados.
 - Usa flash para mostrar mensajes de error en caso de fallos.
 - Retorna True si la operación fue exitosa, y False en caso contrario.



3. Método autenticar_usuario

Verifica las credenciales del usuario para autenticación.

- Funcionamiento:
 - Busca en la tabla usuarios un registro que coincida con el correo y la contraseña proporcionados.
 - Si encuentra un registro, devuelve un diccionario con el ID y el nombre del usuario.
 - Si no encuentra coincidencias o ocurre un error, retorna None y utiliza flash para mostrar el error.

4. Método usuario_existe

Verifica si un correo electrónico ya está registrado en la base de datos.

- Funcionamiento:
 - Ejecuta un comando SQL `SELECT COUNT(*)` para contar los registros con el correo dado.
 - Retorna True si el correo ya existe y False si no.
 - Captura excepciones y utiliza flash para notificar errores.

5. Método crear_usuario

Inserta un nuevo usuario en la base de datos.

- Funcionamiento:
 - Similar a registrar_usuario, pero no verifica si el correo ya existe.



- Usa un comando SQL INSERT INTO para agregar el nuevo usuario.
- Si ocurre un error, imprime el mensaje y realiza un rollback.

6. Estructura de Manejo de Errores

- Todos los métodos implementan manejo de errores con bloques try-except.
- En caso de error:
 - Imprimen o notifican el error (con print o flash).
 - Revierten cambios en la base de datos con rollback.

7. Resumen de Interacción

- Registrar Usuario:
 - Verifica duplicados → Inserta usuario si no existe.
- Autenticar Usuario:
 - Valida credenciales → Retorna datos del usuario si son correctas.
- Actualizar Cita:
 - Modifica detalles de la cita basada en su identificador único (placa).



Calendar_modelo.py

```
import calendar
from datetime import datetime

class CalendarModel:
    def __init__(self):
        self.current_year = datetime.now().year
        self.current_month = datetime.now().month

    def get_calendar(self, year, month):
        return calendar.monthcalendar(year, month)
```

Explicación

1.Propósito:

- Gestionar y obtener información de calendarios mensuales para un año y mes específicos

2.Métodos y Atributos:

- `__init__`:
Inicializa la clase con el año y mes actuales usando la fecha del sistema.
- `get_calendar(year, month)`:
Genera el calendario de un mes específico como una lista de semanas.



Hoy no circula.py

```
from flask import Flask, request, render_template
import re
app = Flask(__name__)
def validar_placa(placa):
    return re.fullmatch(r'[A-Za-z]{3}\d{1,4}', placa) is not None
def hoy_no_circula(placa, dia):
    restricciones = {
        "lunes": [5, 6],
        "martes": [7, 8],
        "miercoles": [4, 3],
        "miércoles": [4, 3],
        "jueves": [1, 2],
        "viernes": [0, 9],
    }
    if dia not in restricciones:
        return None
    ultimo_digito = int(placa[-1]) if placa[-1].isdigit() else None
    if ultimo_digito is None:
        return None
    return ultimo_digito in restricciones[dia]
@app.route('/')
def index():
    return render_template('index.html', message=None)
@app.route('/verificar_hoy_no_circula', methods=['POST'])
def verificar_hoy_no_circula():
    placa = request.form.get('placa')
    if not placa:
        message = "Por favor, proporciona una placa válida."
    elif not validar_placa(placa):
        message = "Por favor, introduce una placa válida (ejemplo: ABC1234).".
    else:
        dia_no_circula = hoy_no_circula(placa)
        if dia_no_circula is None:
            message = "La placa no es válida."
        else:
            message = f"La placa {placa} <strong>NO</strong> puede circular el día {dia_no_circula.capitalize()}."
    return render_template('index.html', message=message)
```



```
if __name__ == "__main__":  
    app.run(debug=True)
```

Explicación

1. Propósito General

El código define una aplicación web con Flask que implementa un sistema para verificar si un vehículo puede circular en un día específico según el esquema "Hoy No Circula".

La aplicación permite a los usuarios:

1. Ingresar una placa y un día.
2. Determinar si el vehículo puede circular o no ese día, basándose en el último dígito de la placa y las restricciones predefinidas.

2. Configuración Inicial

- Se importa y configura Flask.
- El sistema utiliza una función de validación de placas y una lógica específica para determinar los días de restricción de circulación.

3. Validación de Placa

- Comprueba que las placas ingresadas sigan el formato correcto (3 letras seguidas de 1 a 4 dígitos, como "ABC1234").
- Expresión Regular:
 [A-Za-z]{3}: Tres letras (mayúsculas o minúsculas).
 \d{1,4}: De uno a cuatro dígitos.
- Salida: True si la placa es válida, de lo contrario False.



4. Lógica "Hoy No Circula"

Entrada:

- placa: Cadena de texto con la matrícula del vehículo.
- dia: Día de la semana (en español, en minúsculas).

Restricciones:

- Define un diccionario con días y los últimos dígitos de placas restringidas.
- Ejemplo: "lunes": [5, 6] significa que los vehículos con terminación en 5 o 6 no circulan los lunes.

Salida:

- True si la placa tiene restricción para el día especificado.
- None si la placa o el día no son válidos.

5. Rutas del Servidor

- Muestra un formulario HTML inicial (index.html).
- El mensaje de validación inicial está vacío (message=None).

b) Ruta para Verificar

- Procesa los datos enviados desde el formulario para verificar restricciones.

Pasos:

1. Obtiene la placa ingresada desde el formulario (request.form.get('placa')).
2. Valida la placa utilizando validar_placa().



3. Evalúa las restricciones con hoy_no_circula().
4. Devuelve un mensaje dinámico a la plantilla index.html con el resultado.

Diseño

Se trabajará en la personalización visual, aplicando estilos, colores y tipografías que refuercen la identidad del sistema y brinden una experiencia agradable al usuario

Bienvenido

[Hoy No Circula](#)

[Registrar Cita](#)

Registrar Cita

```
{% with messages = get_flashed_messages(with_categories=true) %} {% if messages %}
```

```
{% for category, message in messages %}
```

- {{ message }}

```
{% endfor %}
```

```
{% endif %} {% endwith %}
```

Placa:

Confirmar Placa:

Serie:

Confirmar Serie:

Modelo:

Correo Electrónico:



Calendario

Fecha de Cita: 

Calendario

{% for week in calendar_data %} {% for day in week %} {% endfor %} {% endfor %}

Lun Mar Mié Jue Vie Sáb Dom

{{ day if day != 0 else " " }}

Seleccionada:

Verificar "Hoy No Circula"

Placa:

Día:

¿Qué es el programa "Hoy No Circula"?

El programa "Hoy No Circula" es una iniciativa en México que regula la circulación de vehículos en áreas urbanas para reducir la contaminación. Dependiendo del número final de la placa, el día y las reglas del programa, algunos vehículos no pueden circular en ciertas fechas.

[Volver al inicio](#)

Comprobante de Cita

Placa: {{ cita.placa }}

Modelo: {{ cita.modelo }}

Serie: {{ cita.serie }}

Fecha de la Cita: {{ cita.fecha_cita }}

Hora de la Cita: {{ cita.hora_cita }}

Hora del Registro: {{ cita.hora_registro }}



Resultado

Cada sección del sistema está organizada de forma lógica, permitiendo un acceso rápido a las funcionalidades principales como la verificación vehicular, el registro de citas y el calendario interactivo.

Los formularios y botones están cuidadosamente ubicados, con estilos visuales consistentes que facilitan la interacción del usuario. La paleta de colores utilizada es agradable a la vista y refuerza la identidad del sistema, mientras que los textos y etiquetas están diseñados para ser legibles y comprensibles, guiando al usuario en cada paso del proceso.





Registrar Cita

Diciembre 2024						
Domingo	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

Placa:

Confirmar Placa:

Serie:

Confirmar Serie:

Modelo:

Correo Electrónico:

Fecha de la Cita:

Hora de la Cita:

Registrar Cita

Volver al inicio

HOY NO CIRCULA

Verificar "Hoy No Circula"

Placa del vehículo:

Día:

VERIFICAR

¿Qué es el programa "Hoy No Circula"?

El programa "Hoy No Circula" es una iniciativa en México que regula la circulación de vehículos en áreas urbanas para reducir la contaminación ambiental. Dependiendo del número final de la placa, el día y las reglas del programa, algunos vehículos no pueden circular en ciertas fechas.

HOY NO CIRCULA

Lunes

Placa **5 y 6**

Holograma 1 y 2

Martes

Placa **7 y 8**

Holograma 1 y 2

Miércoles

Placa **3 y 4**

Holograma 1 y 2

Jueves

Placa **1 y 2**

Holograma 1 y 2

Viernes

Placa **9 y 0**

Holograma 1 y 2

Vehículos foráneos con placas extranjeras.

- Terminación de placa.
- Sábados 9 a 22 horas
- Lunes - Viernes 9 a 11 horas

Holograma 1

1 y 3 sáb. de mes, impares

Holograma 2

2 y 4 sáb. de mes, pares

Descansa todos los sábados.

VOLVER AL INICIO



Acceso Administrativo

Usuario:

Contraseña:

Iniciar sesión

Volver al inicio

Panel de Administración

«

Diciembre 2024

»

Domingo	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

ir a Registrar Cita - Comprobar

Registros de Citas

Placa	Serie	Modelo	Correo Electrónico	Fecha de Cita	Hora de Cita
alan34	1234	2020	alan@gmail.com	2025-01-15	9:00:00
NKY2923	1234	2024	adasdas@gmail.com	2025-01-01	10:00:00
NKY29234	1234	modelo a	assadas@gmail.com	2024-12-06	14:00:00
alan34	ABCDE2	2024	alan@gmail.com	2024-12-09	14:00:00
alan34	1234	modelo a	asddsfs@gmail.com	2025-01-10	13:00:00
alan34	ABCDE2	2024	adasdas@gmail.com	2024-12-27	15:00:00
alan34	ABCDE2	2024	asddsfs@gmail.com	2025-01-24	14:00:00



BIENVENIDO

INICIAR SESIÓN



HOY NO CIRCULA



REGISTRAR CITA



ADMINISTRATIVO



Iniciar Sesión o Crear Cuenta

¿No tienes cuenta? [Crea una cuenta](#)

Nombre de usuario

MARCO

Correo electrónico

nieto@gmail.com|

Contraseña

.....

Crear Cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

[Volver al inicio](#)



Iniciar Sesión o Crear Cuenta

Usuario

MARCO

Contraseña

.....

Iniciar Sesión

¿No tienes cuenta? **Crea una cuenta**

Volver al inicio



Registros de Citas

Placa	Serie	Modelo	Correo Electrónico	Fecha de Cita	Hora de Cita	Acción
alan34	1234	2024	alan@gmail.com	2025-01-15	13:43:00	Modificar
NKY2923	1234	2024	adasdas@gmail.com	2025-01-01	10:00:00	Modificar
NKY29234	1234	modelo a	assadas@gmail.com	2024-12-06	14:00:00	Modificar
alan34	1234	2024	alan@gmail.com	2025-01-15	13:43:00	Modificar
alan34	1234	2024	alan@gmail.com	2025-01-15	13:43:00	Modificar
alan34	1234	2024	alan@gmail.com	2025-01-15	13:43:00	Modificar



Modificar Cita

Placa:

Serie:

Modelo:

Correo Electrónico:

Fecha de Cita:



Hora de Cita:



Actualizar Cita

Volver a los registros

Rol de cada integrante

Alan e Ivanna:

Diseñaron la interfaz gráfica del sistema, trabajando en la distribución visual de los elementos, la estética, y la experiencia de usuario para garantizar que sea intuitiva y funcional.

Jorge:

Se encargó del desarrollo del apartado referente al programa "Hoy No Circula". Implementó la lógica necesaria para mostrar correctamente las restricciones vehiculares basadas en los días, colores de engomado y terminación de placas.

Emanuel:



Se encargo del desarrollo en el módulo del calendario, logrando que se visualice correctamente dentro de la aplicación. Además, implementaste la funcionalidad para que los usuarios puedan seleccionar días específicos, permitiendo que se registre su elección en el sistema para futuros usos

Ezequel:

Implementaron la conexión entre la base de datos y las distintas interfaces del sistema. Esto incluyó asegurar que los datos se almacenen, recuperen y actualicen correctamente según las interacciones del usuario.

PythonAnywhere

Enlace

<https://emma12.pythonanywhere.com>



BIENVENIDO

[INICIAR SESIÓN](#)

HOY NO CIRCULA



REGISTRAR CITA



ADMINISTRATIVO



BIENVENIDO



HOY NO CIRCULA



REGISTRAR CITA



ADMINISTRATIVO



GitHub

Se muestra una lista de colaboradores asociados a un proyecto o repositorio, en la plataforma de desarrollo colaborativo como GitHub. Cada colaborador tiene un identificador de usuario único, junto con una etiqueta que los designa como *Colaborador*.

☐		Alancill01 Colaborator	
☐		Iker2x Colaborator	
☐		ivannarosasya Colaborator	
☐		NURIT ESTRELLA niahhjk • Colaborator	
☐		Uri890 Colaborator	

https://github.com/Hakotaroo/P_interfaz



Conclusión

Este proyecto representa una solución integral y eficiente para la gestión vehicular, ofreciendo herramientas prácticas que combinan funcionalidad, diseño intuitivo y manejo robusto de datos. A través de la aplicación web desarrollada con Flask, se integran características clave como la verificación vehicular, el registro de citas y la generación de calendarios, asegurando que los usuarios puedan acceder a toda la información necesaria de manera rápida y sencilla.

La implementación de una base de datos confiable permite la gestión segura y eficiente de la información, mientras que la lógica personalizada garantiza la precisión en la validación de restricciones y la planificación de actividades.

En conjunto, este sistema no solo facilita la organización y el cumplimiento de las normativas vehiculares, sino que también proporciona una experiencia de usuario óptima, convirtiéndose en una herramienta práctica y accesible para todos.